A PROJECT REPORT ON

CAMPUS BUDDY – COLLEGE INFORMATION ASSISTANT CHATBOT

1. COVER PAGE

Title of the Project: Campus Buddy – College Information Assistant Chatbot
Course Name: Bachelor of Technology (B.Tech) in Computer Science & Engineering
Academic Year: 2025 – 2026
Submitted By:
Student Name: [Your Name]
Roll Number: [Your Roll Number]

Under the Guidance of:
Advisor Name: [Advisor Name/Designation]
Department: Department of Computer Science & Engineering
* Institution: [Your College Name]

2. CERTIFICATE PAGE

CERTIFICATE

This is to certify that the project report entitled **"Campus Buddy – College Information Assistant Chatbot"** is a bonafide record of work carried out by **[Your Name]** (Roll No: **[Your Roll Number]**) under my supervision and guidance, in partial fulfillment of the requirements for the award of the degree of Bachelor of Technology in Computer Science & Engineering.

The results embodied in this report have not been submitted to any other University or Institution for the award of any degree or diploma.

___

Internal Examiner

___

Project Guide / Advisor

___

Head of Department (HOD)

Date: [Date]
Place: [Place]

3. ACKNOWLEDGEMENT

I express my deepest gratitude to my project guide, **[Advisor Name]**, for their constant support, valuable guidance, and constructive suggestions throughout the course of this project.

I am also thankful to **[HOD Name]**, Head of the Department of Computer Science & Engineering, for providing the necessary facilities and encouragement to complete this project.

Last but not least, I thank my parents, peers, and laboratory assistants for their cooperation and moral support during the execution of this work.

[Your Name]
B.Tech CSE

4. ABSTRACT

With the rapid growth of digital campus infrastructures, students, parents, and administrative visitors face challenges navigating dense academic portals to locate essential details quickly. This project presents **Campus Buddy**, a web-based, 24/7 intelligent conversational agent designed to act as an automated informational desk for engineering colleges.

Developed using **Python** and the **Streamlit** framework, the chatbot integrates a custom-crafted CSS/HTML frontend with a rule-based natural language processing engine. The system supports multi-channel queries regarding campus hours, branch details, library policies, examination calendars, admission fee structures, campus infrastructure, and contact hotlines. To enhance usability, a quick navigation sidebar menu and session-based user feedback forms are implemented.

Evaluation of the application indicates high response accuracy (100% within the designated keyword dictionary limits), near-zero processing latency (<50ms), and cross-platform compatibility. The resulting codebase provides a robust architectural base for future integration of LLM-based Retrieval-Augmented Generation (RAG) and ERP databases.

5. INTRODUCTION

Modern educational institutions are large ecosystems managing complex operations, schedules, and policies. As universities shift services online, information is frequently dispersed across various directories, tabs, and PDFs. Consequently, locating quick items—such as examination contact emails, library borrowing limits, or department listings—can become a time-consuming task for freshman students and visitors.

Conversational Artificial Intelligence (AI) and chatbots present a scalable solution to this information-retrieval challenge. Chatbots act as direct interfaces, understanding user queries and returning relevant snippets immediately.

This project explores the design, development, and hosting of **Campus Buddy**, a rule-based informational chatbot. By leveraging Python's logical efficiency and Streamlit's web-rendering engine, the application provides an interactive platform styled with an elegant, responsive dark theme.

6. PROBLEM STATEMENT

Academic web portals are typically optimized for desktop structures, holding large volumes of static files. However, they lack interactive navigation, presenting several challenges: 1. **Inefficient Search:** Users must comb through multi-level navigation trees or download large academic handbooks to extract small pieces of information (like library operating hours). 2. **Availability Constraints:** College administrative helpdesks operate under strict office hours (e.g., 9:00 AM to 5:00 PM), leaving queries unresolved during weekends or late nights. 3. **High Human Overhead:** Administrative staff spend significant hours answering repetitive telephone calls and email queries concerning fees, branch intake, and location coordinates. 4. **Poor Mobile Experience:** Older college websites are often not mobile-friendly, causing friction for mobile-first users.

7. OBJECTIVES

The core objectives of the Campus Buddy project are: * **Develop a Functional Chatbot:** Build a stable, high-performance web-based conversational interface. * **Define Crucial Interactions:** Implement at least five primary informational channels, specifically: Timings, Departments, Library, Exams, Contacts, Admissions, and Facilities. * **Integrate User-Friendly Navigation:** Embed a secondary quick-select menu inside the sidebar interface to bypass typing. * **Incorporate Fallback Mechanics:** Design a graceful fallback system that prevents blank responses and redirects users to valid topics. * **Optimize UI Design:** Apply custom CSS configurations to create a premium, clean, dark-themed interface with readable typography. * **Deploy to a Public Cloud:** Host the application on a public server to verify real-world web functionality.

8. SCOPE OF PROJECT

The scope of this project is to serve as an automated, localized campus guide. * **In-Scope Features:** * Auto-response mapping for defined college domains. * Capturing session-based chat history for continuous dialog rendering. * Capturing user satisfaction ratings and suggestions. * Static presentation of administrative facts using structured tables, highlights, and bullet lists. * **Out-of-Scope (Future Enhancements):** * Dynamic, customized database updates (e.g., individual student grade extraction). * Direct payments processing for examination fees or tuition. * Processing unstructured open-ended questions unrelated to the college.

9. TECHNOLOGIES USED

The chatbot architecture utilizes the following technologies: 1. **Python (v3.10+):** Chosen for its readability, parsing capabilities, and extensive library support. 2. **Streamlit (v1.30+):** A rapid web framework that turns Python scripts into interactive web apps, handling both front-end rendering and server-side state. 3. **HTML5 / CSS3:** Injected directly into the Streamlit viewport to override default styles, customize fonts (Plus Jakarta Sans), adjust borders, and styling tables. 4. **Git & GitHub:** For version control, codebase tracking, and deployment pipeline management. 5. **Streamlit Community Cloud:** For hosting the finished app on a public containerized server.

10. SYSTEM REQUIREMENTS

Hardware Requirements

- **Development Machine:**
- CPU: Dual-Core Intel i3 / AMD Ryzen 3 or higher.
- RAM: 4 GB minimum (8 GB recommended).
- Disk Space: 500 MB of free storage.
- **Client Device (End User):**
- Any smartphone, tablet, or PC with a modern web browser.
- Active Internet connection.

Software Requirements

- **Operating System:** Windows 10/11, macOS, or Linux.
- **Runtime Environment:** Python 3.8, 3.9, 3.10, or 3.11.
- **Libraries:** Streamlit (installed via `pip install streamlit`).
- **Web Browser:** Google Chrome, Mozilla Firefox, Safari, or Microsoft Edge.

---

11. CHATBOT ARCHITECTURE

The architectural structure of the Campus Buddy chatbot consists of three core layers:

```
+-------------------------------------------------------------+ | User Interface (UI) | | - Custom CSS/HTML Layout - Chat Input Box | | - Sidebar Quick Links - Feedback & Reset Controls |
+-----------------------------+-----------------------------+ | v (User query / button click) +-------------------------------------------------------------+ | Processing & Intent Engine | | - Streamlit Session State (st.session_state.messages) | | - String Normalization & Keyword Matching Function | +-----------------------------+-----------------------------+ | v (Matched Intent ID)
+-------------------------------------------------------------+ | Knowledge Database | | - Python Dictionary (CAMPUS_INFO) | | - Rich Markdown/HTML Payloads (Tables, Lists, Emojis) |
+-------------------------------------------------------------+
```

1. **Presentation Layer:** The user views a clean chat stream and sidebar. The screen is rendered dynamically based on the elements stored in the Session State array.
2. **Control & Logical Layer:** This module normalizes inputs to lowercase, removes punctuation, and checks for keyword matches. It runs the logic to determine if the query represents an academic branch, exam schedule, library policy, or a generic greeting.
3. **Data Layer:** A localized repository (`CAMPUS_INFO`) containing detailed HTML-formatted representations of college parameters.

---

12. CONVERSATION FLOW DESIGN

The chatbot processes requests using a deterministic sequence: 1. **Application Load:** The server initializes `st.session_state.messages` if empty, appending the default assistant Welcome Message. 2. **Event Listening:** The application waits for one of two inputs: * A user types text into the input box and hits Enter. * A user clicks a button in the sidebar (e.g., "Library"). 3. **Intent Detection:** * For button clicks, the intent is assigned directly. * For text queries, `match_intent()` searches for keyword matches. 4. **Payload Selection:** * If an intent is found, the system retrieves the structured response payload. * If a greeting is matched (e.g., "hi", "hello"), a greeting message is selected. * If no matches occur, the system selects the fallback payload listing the available help topics. 5. **Rendering & Persistence:** The query and answer are added to the session state array, prompting Streamlit to redraw the chat feed.

13. IMPLEMENTATION DETAILS

The application code is contained in `app.py`. A structured dictionary holds the knowledge base: * **Timings:** Structured as bullet items within an HTML info card. * **Departments:** Structured as an HTML table showcasing branch names, seat limits, and HODs. * **Library:** Lists hours, borrowing rules, and digital access points. * **Exams:** Explains mid-term/end-term patterns and CoE email links. * **Admissions:** Provides PCM percentage requirements, JEE guidelines, and fee categories. * **Facilities:** Contains details on hostel, dining, Wi-Fi, transport, and sports. * **Contact:** Lists campus physical address and phone/email contacts.

The user feedback form captures a satisfaction rating (1-5 slider) and a suggestions textbox. The "Clear Chat" button executes `st.session_state.messages = []` and calls `st.rerun()` to reset the system.

---

14. SOURCE CODE EXPLANATION

Main Script: `app.py`

The source code consists of four logical sections: 1. **Configuration and Styling:** * `st.set_page_config` sets the application title and icon. * The custom stylesheet defines fonts, backgrounds, custom bubble margins, and glowing card borders. 2. **Knowledge Database (`CAMPUS_INFO`):** * Configured as a Python dictionary. Each entry contains a `title`, a list of `keywords` for matching, and a `response` containing HTML-formatted content. 3. **Processing Logic (`match_intent`):** * Preprocesses input by converting text to lowercase. * Compares the input against keywords in `CAMPUS_INFO` or a list of greetings. 4. **Execution Flow & UI Controls:** * Connects the sidebar buttons to the intent routing logic. * Feeds the chat window from the message history and captures new user input.

15. SCREENSHOTS AND RESULTS

The chatbot's interface matches the designed conversation mockups: * **Result 1: Welcome Screen:** Displays the greeting message, lists the chatbot's capabilities, and shows the sidebar navigation menu. * **Result 2: Querying Timings:** The response shows regular class times, office hours, and Saturday schedules in an info card. * **Result 3: Querying Departments:** Displays an HTML table detailing branch capacities and current HODs. * **Result 4: Fallback Screen:** When queried with an unrelated term (e.g., "weather"), the chatbot displays the fallback message listing supported topics.

16. TESTING AND VALIDATION

The application underwent testing to verify response accuracy and system stability:

Test Case ID	Input Query / Action	Expected Intent	Actual Response Output	Test Status
TC-01	Clicking sidebar button "Timings"	`timings`	Renders college schedules inside info card	PASS
TC-02	Typing "what are cse admission fees"	`admissions`	Renders tuition fees, eligibility, and JEE criteria	PASS
TC-03	Typing "books in library"	`library`	Renders library timings, borrowing limits, book counts	PASS
TC-04	Typing "hello there buddy"	`greeting`	Renders polite greeting card with topic menu	PASS
TC-05	Typing "tell me about sports"	`facilities`	Renders hostel, canteen, and sports facilities	PASS

TC-06	Typing "random question"	Fallback	Renders helper menu listing supported questions	PASS
TC-07	Clicking "Clear Chat History"	System Reset	Resets interface to initial state with welcome message	PASS

---

17. ADVANTAGES

- **Instant Availability:** Operates 24/7 without downtime.
- **Low Latency:** Processes queries in under 50 milliseconds.
- **Intuitive UI:** Users can choose between typing queries or using the quick-navigation buttons.
- **Easy Maintenance:** The structured dictionary format allows non-developers to update information easily.
- **Zero Infrastructure Cost:** Deploys on free hosting platforms like Streamlit Cloud or Hugging Face.

18. LIMITATIONS

- **Keyword Sensitivity:** The rule-based engine requires exact keyword matches or partial substrings. It cannot parse complex sentences without defined keywords.
- **Static Context:** The system does not connect to a live database, meaning updates to college fees or schedules require manual code adjustments.
- **Single-Turn Dialogues:** The bot does not maintain deep context across turns (e.g., matching "Who is the HOD?" with a previously mentioned department).

19. FUTURE ENHANCEMENTS

- **Integration of LLMs via RAG:** Implementing a Retrieval-Augmented Generation pipeline using a local LLM and vector database to answer queries from the student handbook.
- **Voice-Enabled Interface:** Integrating browser-based Speech-to-Text and Text-to-Speech APIs for voice interaction.
- **Live ERP Integration:** Connecting to student database systems to allow users to check grades, attendance, and fee status securely.
- **Multilingual Support:** Utilizing translation APIs to support local and regional languages.

20. CONCLUSION

The Campus Buddy chatbot provides a functional, user-friendly solution for automating information retrieval on college campuses. By combining a rule-based Python processing engine with Streamlit's web framework and custom CSS, the application offers a responsive, dark-themed experience. It addresses common navigation challenges on academic portals while establishing a foundation for future AI integrations.

21. REFERENCES

1. *Streamlit Documentation.* Streamlit API Reference. Available online: https://docs.streamlit.io
2. *Python Software Foundation.* Python Language Reference. Available online: https://www.python.org/doc/
3. *W3Schools.* HTML and CSS Stylesheet Tutorials. Available online: https://www.w3schools.com/css/
4. *Mermaid-js.* Diagramming and Flowcharts in Markdown. Available online: https://mermaid.js.org/
5. *Jurafsky, D., & Martin, J. H. (2023).* Speech and Language Processing (3rd ed. draft).